NET-NEUTRAL AI

Technical Requirements Document

Decentralized Federated Learning Platform

GitHub DevDays Hackathon 2026 | Version 1.0 | May 2026

Document Owner	CS Team Member (Janhvesh)
Companion Doc	Net-Neutral AI PRD v1.0
Status	Draft — v1.0
OS Target	Windows 10/11 (all machines)
Build Window	May 25 – June 10, 2026 (16 days)
Demo Format	Recorded video — 4 tiled terminals on one screen

1. Hardware Allocation

Four machines are available. Roles are assigned based on capability, not preference.

Machine	Owner	Specs	Role	Runs What
Machine A	ENTC1	GPU laptop (dedicated GPU)	Coordinator + Client A	Flask server + training client
Machine B	Janhvesh (CS)	Decent CPU laptop	Client B	Training client only
Machine C	3rd member	Decent CPU laptop	Client C	Training client only
Machine D	4th member	i3, 4–8 GB RAM	Coordinator fallback / observer	Flask server only if ENTC1 unavailable

Why Machine A runs both coordinator and Client A

The coordinator (Flask server) uses almost no compute — it just averages weights and writes to SQLite. Running it alongside training on the GPU laptop adds negligible overhead. This keeps Machine D free as a pure fallback and avoids wasting the GPU.

1.1 Machine D — Two Scenarios

The TRD accounts for both possibilities depending on Machine D's actual performance on demo day.

Scenario A — Machine D can train	Scenario B — Machine D cannot train
Machine D joins as Client D (4th node)	Machine D runs coordinator only, no training
Coordinator moves to a standalone process on Machine A	Machine A = coordinator + Client A (default plan)
Demo shows 4 clients + 1 coordinator	Demo shows 3 clients + 1 coordinator
More impressive visually	Safer and simpler — recommended default
Use only if tested successfully on Machine D before demo day	This is the plan you build toward first

2. Model & Dataset Selection

Why not DistilBERT for the prototype

DistilBERT fine-tuning takes 3–8 minutes per round on a CPU laptop. With 5 rounds across 3 clients, that is potentially 40+ minutes of recording time. A custom lightweight Transformer

does the same job architecturally, trains in 20–40 seconds per round on CPU, and is honest — you are demonstrating the federated system, not the model.

2.1 Model Specification

Parameter	Specification	Reason
Architecture	2-layer Transformer classifier	CPU-trainable, architecturally identical to DistilBERT in principle
Embedding dim	128	Small enough for CPU, large enough to learn
Attention heads	4	Minimum for meaningful multi-head attention
Hidden dim (FFN)	256	Keeps parameter count under 2M total
Total parameters	~1.5–2M	Trains in 20–40s per round on i5/i7 CPU
Vocabulary size	10,000 tokens	Built from dataset using simple tokeniser
Max sequence length	128 tokens	Covers 95% of short text samples
Output classes	2 (positive / negative)	Binary sentiment classification

2.2 Dataset Specification

Parameter	Specification
Dataset	IMDb Movie Reviews (public domain, HuggingFace datasets library)
Total samples used	15,000 (subset of full 50K for speed)
Split per client	5,000 samples each — IID (evenly balanced positive/negative)
Validation set	2,000 samples held on coordinator for global evaluation only
Format	Plain text CSV — tokenised at runtime by each client
Download method	datasets library: load_dataset('imdb') — done once at setup

2.3 Training Hyperparameters

Hyperparameter	Value	Notes
Optimiser	Adam	AdamW if time allows — Adam is safer for beginners
Learning rate	1e-3	Higher than DistilBERT because training from scratch

Epochs per round	2	Balance between learning and round speed
Batch size	32	Safe for 4GB RAM machines
Training rounds	5	Enough to show clear accuracy trend in video
Loss function	CrossEntropyLoss	Standard for classification
Evaluation metric	Accuracy (%)	Simple, visually clear for judges

3. System Components & File Structure

3.1 Repository Structure

```
net-neutral-ai/
├── coordinator/
│   ├── server.py          # Flask coordinator — main entry point
│   ├── fedavg.py          # FedAvg averaging logic
│   ├── credits.py         # Credit ledger — SQLite read/write
│   ├── database.db        # SQLite database (auto-created on first run)
│   └── requirements.txt   # Flask, torch, numpy
├── client/
│   ├── client.py          # Client entry point — runs training loop
│   ├── model.py           # Transformer model definition
│   ├── train.py           # Local training logic
│   ├── data.py            # Dataset loading and tokenisation
│   └── requirements.txt   # torch, requests, datasets
├── shared/
│   └── config.py          # Coordinator IP, port, round count — edit before demo
├── demo/
│   └── run_demo.bat       # Windows batch script — launches all 4 terminals at
once
├── .github/
│   └── workflows/
│       └── lint.yml       # GitHub Actions — basic Python lint (satisfies
hackathon req)
└── README.md              # Setup instructions, architecture diagram, how to run
```

3.2 config.py — The Only File You Edit Before Demo

```
# shared/config.py
# Edit COORDINATOR_IP before every demo session

COORDINATOR_IP   = '192.168.1.X'   # Replace X with Machine A's local IP
COORDINATOR_PORT = 5000
TOTAL_ROUNDS     = 5
LOCAL_EPOCHS     = 2
BATCH_SIZE       = 32
LEARNING_RATE    = 1e-3
DATA_PATH        = './data/'        # Local data shard folder for this client
```

4. Coordinator Technical Specification

4.1 Responsibilities

- Serve the current global model weights to any registered client on request
- Receive weight updates from clients after each local training round
- Run FedAvg: compute element-wise mean of all received state_dicts
- Evaluate the new global model on the held-out validation set
- Log credits to SQLite after each round
- Print round summary to terminal after all clients submit
- Handle a missing client gracefully — average whoever submitted, skip the rest

4.2 API Endpoints

Endpoint	Method	Request Body	Response Body	Owner
/register	POST	{ "client_id": "client_A" }	{ "status": "ok", "round": 1 }	IT
/model	GET	None	Binary file — model weights via torch.save	IT
/submit	POST	Multipart: weights file + JSON metadata	{ "credits": 42, "round": 2, "global_acc": 0.71 }	IT + CS
/status	GET	None	{ "round": 3, "clients": [...], "leaderboard": [...] }	IT
/results	GET	None	Full accuracy history across all rounds as JSON	IT

4.3 FedAvg Algorithm Specification

The FedAvg function lives in coordinator/fedavg.py. It accepts a list of state_dicts and returns one averaged state_dict.

```
# fedavg.py — specification (not implementation)

Input:  List of PyTorch state_dicts from N clients
        e.g. [state_dict_A, state_dict_B, state_dict_C]

Output: One averaged state_dict

Algorithm:
  For each key (layer name) in the state_dict:
    averaged_weight[key] = mean of all client weights at that key
    i.e. (client_A[key] + client_B[key] + client_C[key]) / N

  Load averaged_weight into global model
  Evaluate global model on validation set
```

```
  Return global_accuracy

Edge case: if a client did not submit this round, exclude it from N
  i.e. N = number of clients who actually submitted, not total registered
```

4.4 Credit Calculation

Input	Formula	Example
samples_trained (integer)	points = floor(samples_trained / 5)	5000 samples = 1000 points per round
Cumulative credits	Sum of points across all rounds	5 rounds x 1000 = 5000 total points
Leaderboard sort	Descending by cumulative credits	Client A: 5000 | Client B: 4800 | Client C: 5100

5. Client Technical Specification

5.1 Client Startup Sequence

1. Read COORDINATOR_IP and config values from shared/config.py
2. POST to /register with client_id (e.g. 'client_A')
3. GET /model — download current global model weights binary file
4. Load weights into local model instance via torch.load
5. Load local data shard from DATA_PATH
6. Begin training loop for TOTAL_ROUNDS rounds

5.2 Per-Round Training Loop

7. Train local model for LOCAL_EPOCHS epochs on local data shard
8. Record: samples_trained, time_seconds, final local loss
9. Save model weights to a temp file using torch.save
10. POST to /submit — send weights file + metadata as multipart form
11. Receive response: credits_earned, round number, global_acc
12. Print round summary to terminal
13. Wait for coordinator signal or fixed delay before next round

5.3 Terminal Output Format

Every client prints this format after each round. This is what appears in the demo video.

```
==========================================
 Net-Neutral AI   |   CLIENT A   |   Round 3 / 5
==========================================
 Local training : COMPLETE
 Samples trained: 5000
 Local loss     : 0.4821
 Time taken     : 28.3 seconds
------------------------------------------
 Credits earned : 1000
 Total credits  : 3000
 Global accuracy: 74.2%
==========================================
 Waiting for next round...
```

6. ML Model Technical Specification

This section is Janhvesh's primary build target. It specifies exactly what model.py and train.py must implement.

6.1 Model Architecture — client/model.py

Component	What It Does	Key Detail
Embedding layer	Converts token IDs to vectors	vocab_size=10000, embed_dim=128
Positional encoding	Adds position info to embeddings	Simple learned positional embedding, max_len=128
TransformerEncoder (x2)	2 stacked self-attention layers	nhead=4, dim_feedforward=256, dropout=0.1
Global avg pooling	Collapses sequence to single vector	Mean across sequence length dimension
Linear classifier	Maps vector to 2 class scores	Linear(128, 2)
Forward pass output	Raw logits for 2 classes	Shape: (batch_size, 2)

6.2 Data Pipeline — client/data.py

Step	Specification
Load dataset	Use HuggingFace datasets: load_dataset('imdb') — done once, cached locally
Select shard	Each client receives a different index range: A=0:5000, B=5000:10000, C=10000:15000
Tokenise	Simple whitespace tokeniser — build vocab from training split, map words to integer IDs

Pad / truncate	All sequences padded or truncated to max_len=128
DataLoader	torch.utils.data.DataLoader with batch_size=32, shuffle=True
Validation set	2000 samples (indices 45000:47000) — held only on coordinator, never on clients

6.3 Weight Serialisation — how weights travel

This is the bridge between the ML pipeline and the network layer. Get this right and everything connects.

```
# CLIENT SIDE — saving weights to send
import torch, tempfile, os

def save_weights(model):
    tmp = tempfile.NamedTemporaryFile(delete=False, suffix='.pt')
    torch.save(model.state_dict(), tmp.name)
    return tmp.name   # pass this path to the POST request

# COORDINATOR SIDE — loading received weights
def load_weights(file_path, model):
    state_dict = torch.load(file_path, map_location='cpu')
    model.load_state_dict(state_dict)
    return model
```

7. Network Technical Specification

7.1 Communication Protocol

Property	Decision	Reason
Protocol	HTTP over local WiFi	Simple, no firewall config, works on Windows out of the box
Framework	Flask (coordinator) + requests library (client)	Both teams already have some familiarity
Weight transfer	Multipart form-data (binary file upload)	torch.save binary is not JSON-safe — file upload is the clean solution
Metadata transfer	JSON fields in the same multipart POST	Keeps weight upload and metadata in one request
Port	5000 (Flask default)	No conflicts with standard Windows services
Coordinator IP	Static local IP set in config.py before demo	Avoids any DNS or discovery complexity

7.2 Pre-Demo Network Checklist

- All machines connected to same WiFi network
- Run ipconfig on Machine A — note IPv4 address — paste into shared/config.py
- Disable Windows Defender Firewall for private networks on Machine A (or add port 5000 exception)
- Test: from Machine B, open browser and go to http://[Machine A IP]:5000/status — must return JSON
- If no response: check firewall first, then check that Flask is running with host='0.0.0.0'

Critical Flask Startup Flag

The coordinator must run with: app.run(host='0.0.0.0', port=5000). The default host='127.0.0.1' only accepts connections from the same machine. Without host='0.0.0.0', clients on other laptops will time out silently.

8. Demo Video Specification

The demo video is the final deliverable. Every technical decision in this document serves this recording.

8.1 Recording Setup

Property	Specification
Recording machine	Machine A (ENTC1's GPU laptop) — most powerful, smoothest recording
Layout	4 terminals tiled on one screen: coordinator top-left, Client A top-right, Client B bottom-left, Client C bottom-right
Terminal emulator	Windows Terminal (supports multiple panes) — install if not already present
Screen resolution	1920x1080 minimum — font size 14+ so terminals are readable in video
Recording software	OBS Studio (free) or Windows built-in Xbox Game Bar (Win+G)
Video length	Target 3–5 minutes — 5 rounds of training + brief status/leaderboard at end
Audio	Optional voiceover explaining each round — adds polish but not required

8.2 Demo Script — What Happens On Screen

Timestamp	What Happens	Terminal Output Visible

0:00 – 0:20	Coordinator starts, prints 'Server running on 0.0.0.0:5000'	Coordinator terminal
0:20 – 0:40	3 clients start, each prints 'Registered with coordinator. Downloading model...'	All 3 client terminals
0:40 – 2:00	Round 1: all 3 clients train simultaneously, print local loss	3 client terminals active
2:00 – 2:20	Coordinator receives all 3 updates, prints 'Round 1 complete. Global accuracy: 61.2%'	Coordinator terminal
2:20 – 4:00	Rounds 2–4 repeat — accuracy visibly climbs each round	All 4 terminals
4:00 – 4:30	Round 5 complete — coordinator prints final leaderboard with credits	Coordinator terminal
4:30 – 5:00	curl /status or browser hit shows final JSON — accuracy, credits, rounds	Browser or terminal

9. Dependencies & Environment Setup

9.1 Coordinator — requirements.txt

```
flask==3.0.3
torch==2.3.0          # CPU version sufficient for coordinator
numpy==1.26.4
```

9.2 Client — requirements.txt

```
torch==2.3.0          # CPU version for Machines B and C
                      # GPU version (torch+cuda) for Machine A
requests==2.32.3
datasets==2.19.0      # HuggingFace — for IMDb download
numpy==1.26.4
```

9.3 Setup Commands — Windows

```
# Run once on each machine
python -m venv venv
venv\Scripts\activate

# On coordinator machine (Machine A):
pip install -r coordinator/requirements.txt

# On client machines (Machines B and C):
pip install -r client/requirements.txt
```

```
# Download dataset once (run on each client machine):
python -c "from datasets import load_dataset; load_dataset('imdb')"
```

10. Technical Risk Registry

Risk	Impact	Mitigation	Owner
College WiFi blocks device-to-device traffic	High — kills demo entirely	Use personal hotspot or home WiFi for recording. Already planned.	ENTC1
Training too slow on CPU laptops	High — demo drags or times out	Custom small model (2M params) trains in ~30s on CPU. Test on day 1.	CS (Janhvesh)
Windows Firewall blocks port 5000	Medium — clients cannot reach coordinator	Add firewall exception or disable for private networks before demo.	IT
Machine D cannot handle training	Low — already accounted for	Scenario B in section 1.1 — Machine D runs coordinator only.	All
Model accuracy does not improve	Medium — demo looks broken	Use IID data split. Convergence is near-guaranteed. Test in week 1.	CS (Janhvesh)
torch.save file corrupted in transfer	Medium — FedAvg crashes	Add try/except on coordinator — skip corrupted client, log error.	IT + CS
A client crashes mid-round	Low — FedAvg handles missing client	Coordinator averages whoever submitted. Spec already handles this.	IT

11. Module Handoff Contracts

These are the exact agreements between team members. If your module satisfies its contract, the system works. Do not change these without team agreement.

CS → IT (Janhvesh to IT teammate)

client/model.py exports a class named TransformerClassifier. It has a standard PyTorch state_dict. IT's coordinator calls torch.load on the received file and gets a valid state_dict back. No other assumptions.

IT → CS (IT teammate to Janhvesh)

The /model endpoint returns a binary .pt file. Janhvesh's client does torch.load(downloaded_file, map_location='cpu') and gets a state_dict compatible with TransformerClassifier. The /submit endpoint accepts a multipart POST with fields: 'weights' (file), 'client_id' (string), 'samples' (int), 'time_seconds' (float).

ENTC → CS + IT (Client app to both)

client.py is the entry point. It imports from model.py and train.py (CS) and posts to the coordinator (IT). ENTC builds the loop and networking glue. Any change to the POST format or model import must be communicated to ENTC immediately.

Appendix: Technology Decisions Log

Decision	Chosen	Rejected	Reason
Model	Custom 2-layer Transformer	DistilBERT	DistilBERT too slow for CPU live demo
Weight transfer	torch.save binary file upload	JSON serialisation of weights	JSON of float arrays is ~4x larger and slower
Credit system	SQLite	Blockchain / Solidity	Blockchain is 3 months of work. SQLite is 1 hour.
Communication	Flask + requests	gRPC	gRPC requires protobuf setup — unnecessary complexity
P2P gossip	Not in prototype	libp2p	Out of scope. Central coordinator is sufficient for demo.
Demo format	Recorded video	Live presentation	Hackathon requires video submission — recorded run is safer
OS	Windows only	Cross-platform	All team machines are Windows — no point abstracting